Lab-01 (DBMS)

FRIDAY, JANUARY 17, 2025

NED University
Name: MUHAMMAD OBAID
Roll Number : CT-23025

Exercise:

1. Display the last name concatenated with the job ID, separated by a comma and space, and name the column Employee and Title.

Query:

```
SELECT LAST_NAME || ' , ' || JOB_ID AS "Employee and Title" FROM hr.employees ;
```

Output:

Employee and Title
Abel , SA_REP
Ande , SA_REP
Atkinson , ST_CLERK
Austin , IT_PROG
Baer , PR_REP
Baida , PU_CLERK
Banda , SA_REP

2. Create a query to display all the data from the EMPLOYEES table. Separate each column by a comma. Name the column THE_OUTPUT.

Query:

```
SELECT
  EMPLOYEE_ID || ', ' || FIRST_NAME || ', ' || LAST_NAME || ', ' || EMAIL || ', ' ||
    PHONE_NUMBER || ', ' || HIRE_DATE || ', ' || JOB_ID || ', ' || SALARY ||
    COMMISSION_PCT || ', ' || MANAGER_ID || ', ' || DEPARTMENT_ID AS "THE_OUTPUT"
FROM hr.employees;
```

Output:

THE_OUTPUT
100, Steven, King, SKING, 515.123.4567, 17-JUN-03, AD_PRES, 24000, , 90
101, Neena, Kochhar, NKOCHHAR, 515.123.4568, 21-SEP-05, AD_VP, 17000, 100, 90
102, Lex, De Haan, LDEHAAN, 515.123.4569, 13-JAN-01, AD_VP, 17000, 100, 90
103, Alexander, Hunold, AHUNOLD, 590.423.4567, 03-JAN-06, IT_PROG, 9000, 102, 60
104, Bruce, Ernst, BERNST, 590.423.4568, 21-MAY-07, IT_PROG, 6000, 103, 60
105, David, Austin, DAUSTIN, 590.423.4569, 25-JUN-05, IT_PROG, 4800, 103, 60
106, Valli, Pataballa, VPATABAL, 590.423.4560, 05-FEB-06, IT_PROG, 4800, 103, 60

3. Show the structure of the DEPARTMENTS table. Select all data from the table.

Query:

```
DESC hr.departments;
SELECT * FROM hr.departments;
```

Output:

TABLE DEPARTMENTS

Column	Null?	Type
DEPARTMENT_ID	NOT NULL	NUMBER(4,0)
DEPARTMENT_NAME	NOT NULL	VARCHAR2(30)
MANAGER_ID	-	NUMBER(6,0)
LOCATION_ID	-	NUMBER(4,0)

Download CSV

4 rows selected.

DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
10	Administration	200	1700
20	Marketing	201	1800
30	Purchasing	114	1700
40	Human Resources	203	2400
50	Shipping	121	1500
60	IT	103	1400
70	Public Relations	204	2700

4. Show the structure of the EMPLOYEES table. Create a query to display the last name, job code, hire date, and employee number for each employee, with employee number appearing first.

Query:

```
DESC hr.employees;
SELECT  EMPLOYEE_ID, LAST_NAME, JOB_ID,
HIRE_DATE FROM hr.employees;
```

Output:

TABLE EMPLOYEES

Column	Null?	Type
EMPLOYEE_ID	NOT NULL	NUMBER(6,0)
FIRST_NAME	-	VARCHAR2(20)
LAST_NAME	NOT NULL	VARCHAR2(25)
EMAIL	NOT NULL	VARCHAR2(25)
PHONE_NUMBER	-	VARCHAR2(20)
HIRE_DATE	NOT NULL	DATE
JOB_ID	NOT NULL	VARCHAR2(10)

SALARY	-	NUMBER(8,2)
COMMISSION_PCT	-	NUMBER(2,2)
MANAGER_ID	-	NUMBER(6,0)
DEPARTMENT_ID	-	NUMBER(4,0)

Download CSV

11 rows selected.

EMPLOYEE_ID	LAST_NAME	JOB_ID	HIRE_DATE
100	King	AD_PRES	17-JUN-03
101	Kochhar	AD_VP	21-SEP-05
102	De Haan	AD_VP	13-JAN-01
103	Hunold	IT_PROG	03-JAN-06
104	Ernst	IT_PROG	21-MAY-07
105	Austin	IT_PROG	25-JUN-05
106	Pataballa	IT_PROG	05-FEB-06
107	Lorentz	IT_PROG	07-FEB-07
108	Greenberg	FI_MGR	17-AUG-02

5. Create a query to display unique job codes from the EMPLOYEES table.

Query:

```
SELECT DISTINCT JOB_ID FROM hr.employees ;
```

Output:

JOB_ID
AC_ACCOUNT
AC_MGR
AD_ASST
AD_PRES
AD_VP
FI_ACCOUNT
FI_MGR
HR_REP
IT_PROG
MK_MAN
MK_REP
PR_REP
PU_CLERK
PU_MAN
SA_MAN
SA_REP
SH_CLERK
ST_CLERK
ST_MAN

Download CSV

19 rows selected.

6. There are four coding errors in this statement. Can you identify them?

SELECT employee_id, last_name sal x 12 ANNUAL SALARY FROM employees;

Errors:

1. Missing comma between last_name and the expression sal x 12.
2. sal should be a column name or an alias**:**
3. Multiplication syntax error i.e. * operator should be used instead of x**.**
4. ANNUAL SALARY should be represented as an alias for computed column.

Corrected Syntax:

```
SELECT employee_id, last_name, SALARY * 12 AS
ANNUAL_SALARY FROM employees;
```

7. Make account on leetcode, head to https://leetcode.com/studyplan/top-sql-50/ and solve 5 questions in the select tag. The questions involve usage of “Where” clause.

You can verify my progress and review the questions I've solved by visiting my LeetCode profile: https://leetcode.com/u/obaid4601261/

Lab-02 (DBMS)

WEDNESDAY, JANUARY 22, 2025

NED University
Name: MUHAMMAD OBAID
Roll Number : CT-23025

Note: The results are not displayed in full due to the large number of rows.

Exercise:

1. Create a query to display the last name and salary of employees earning more than $12,000.

Query:

```
SELECT LAST_NAME, SALARY FROM hr.employees WHERE SALARY > 12000
;
```

Output:

LAST_NAME	SALARY
King	24000
Kochhar	17000
De Haan	17000
Greenberg	12008
Russell	14000
Partners	13500
Hartstein	13000
Higgins	12008

Download CSV

8 rows selected.

2. Create a query to display the employee last name and department number for employee number 176.

Query:

```
SELECT LAST_NAME, DEPARTMENT_ID FROM hr.employees WHERE EMPLOYEE_ID = 176
```

Output:

LAST_NAME	DEPARTMENT_ID
Taylor	80

Download CSV

3. Display the last name and salary for all employees whose salary is not in the range of $5,000 and $12,000.

Query:

```
SELECT LAST_NAME, SALARY FROM hr.employees WHERE SALARY NOT
BETWEEN 5000 AND 12000;
```

Output:

LAST_NAME	SALARY
King	24000
Kochhar	17000
De Haan	17000
Austin	4800
Pataballa	4800
Lorentz	4200
Greenberg	12008

4. Display the employee last name, job ID, and start date of employees hired between February 20, 1998, and May 1, 1998. Order the query in ascending order by start date.

Query:

```
SELECT LAST_NAME, JOB_ID, HIRE_DATE AS START_DATE
FROM hr.employees
WHERE HIRE_DATE BETWEEN TO_DATE('1998-02-20', 'YYYY-MM-DD')
                    AND TO_DATE('1998-05-01', 'YYYY-MM-DD')
ORDER BY HIRE_DATE ;
```

Output:

```
no data found
```

5. Display the last name and department number of all employees in departments 20 and 50 in alphabetical order by name.

Query:

```
SELECT LAST_NAME, DEPARTMENT_ID FROM hr.employees
    WHERE DEPARTMENT_ID = 20 OR DEPARTMENT_ID = 50
ORDER BY LAST_NAME;
```

Output:

LAST_NAME	DEPARTMENT_ID
Atkinson	50
Bell	50
Bissot	50
Bull	50
Cabrio	50
Chung	50
Davies	50

6. List the last name and salary of employees who earn between $5,000 and $12,000, and are in department 20 or 50. Label the columns Employee and MonthlySalary, respectively.

Query:

```
SELECT LAST_NAME AS Employee , SALARY AS Monthly_Salary FROM
hr.employees
    WHERE ( SALARY BETWEEN 5000 AND 12000 )
    AND ( DEPARTMENT_ID BETWEEN 20 AND 50 ) ;
```

Output:

EMPLOYEE	MONTHLY_SALARY
Raphaely	11000
Weiss	8000
Fripp	8200
Kaufling	7900
Vollman	6500
Mourgos	5800
Fay	6000

7. Display the last name and hire date of every employee who was hired in 1994.

Query:

```
-- Oracle doesn't support YEAR(HIRE_DATE)
SELECT LAST_NAME, HIRE_DATE FROM hr.employees
    WHERE EXTRACT(YEAR FROM HIRE_DATE) = 1994 ;
```

Output:

```
no data found
```

8. Display the last name and job title of all employees who do not have a manager.

Query:

```
SELECT LAST_NAME, JOB_ID FROM hr.employees WHERE MANAGER_ID IS
NULL ;
```

Output:

LAST_NAME	JOB_ID
King	AD_PRES

Download CSV

9. Display the last name, salary, and commission for all employees who earn commissions. Sort data in descending order of salary and commissions.

Query:

```
SELECT LAST_NAME, SALARY, COMMISSION_PCT FROM hr.employees
    WHERE COMMISSION_PCT IS NOT NULL
    ORDER BY SALARY DESC , COMMISSION_PCT DESC ;
```

Output:

LAST_NAME	SALARY	COMMISSION_PCT
Russell	14000	.4
Partners	13500	.3
Errazuriz	12000	.3
Ozer	11500	.25
Cambrault	11000	.3
Abel	11000	.3
Vishney	10500	.25

10. Display the last names of all employees where the third letter of the name is an a.

Query:

```
-- Starting 2 can be any prefix and then any/nor suffix str
after 3rd 'a'.
SELECT LAST_NAME FROM hr.employees
    WHERE LAST_NAME LIKE '__a%' ;
```

Output:

LAST_NAME
Grant
Grant
Whalen

Download CSV

3 rows selected.

11. Display the last name of all employees who have an a and an e in their last name

Query:

```
SELECT LAST_NAME FROM hr.employees
    WHERE LAST_NAME LIKE '%a%' AND LAST_NAME LIKE '%e%';
```

Output:

LAST_NAME
Baer
Bates
Colmenares
Davies
De Haan
Faviet
Fleaur

12. Display the last name, job, and salary for all employees whose job is sales representative or stock clerk and whose salary is not equal to $2,500, $3,500, or $7,000.

Query:

```
SELECT LAST_NAME , SALARY, JOB_ID FROM hr.employees
    WHERE JOB_ID IN ('SA_MAN', 'ST_CLERK')
    AND SALARY NOT IN (2500, 3500, 7000) ;
```

Output:

LAST_NAME	SALARY	JOB_ID
Russell	14000	SA_MAN
Partners	13500	SA_MAN
Errazuriz	12000	SA_MAN
Cambrault	11000	SA_MAN
Zlotkey	10500	SA_MAN
Nayer	3200	ST_CLERK
Mikkilineni	2700	ST_CLERK

13. Display the last name, salary, and commission for all employees whose commission amount is 20%

Query:

```
SELECT LAST_NAME, SALARY, COMMISSION_PCT FROM hr.employees
    WHERE COMMISSION_PCT = 0.2 ;
```

Output:

LAST_NAME	SALARY	COMMISSION_PCT
Zlotkey	10500	.2
Olsen	8000	.2
Cambrault	7500	.2
Bloom	10000	.2
Fox	9600	.2
Taylor	8600	.2
Livingston	8400	.2

Lab-03 (DBMS)

WEDNESDAY, JANUARY 29, 2025

NED University
Name: MUHAMMAD OBAID
Roll Number : CT-23025

Note: The results are not displayed in full due to the large number of rows.

Exercise:

1. Write a query that displays the employee's last names all in uppercase and the length of the last names, for all employees whose name starts with S, A, or M. Give each column an appropriate label. Sort the results by the employees' last names

Query:

```
SELECT UPPER(LAST_NAME) AS last_name, LENGTH(LAST_NAME) AS len
FROM hr.employees
WHERE LAST_NAME LIKE 'A%' OR LAST_NAME LIKE 'S%' OR LAST_NAME
LIKE 'M%'
ORDER BY LAST_NAME;
```

Output:

LAST_NAME	LEN
ABEL	4
ANDE	4
ATKINSON	8
AUSTIN	6
MALLIN	6
MARKLE	6
MARLOW	6

2. For each employee, display the employee's last name, and calculate the number of weeks between today and the date the employee was hired. Label the column WEEKS_WORKED . Order your results by the number of WEEKs employed. Round the number of weeks up to the closest whole number.

Query:

```
SELECT LAST_NAME , ROUND( MONTHS_BETWEEN( SYSDATE , HIRE_DATE )
/ 7 ) AS WEEKS_WORKED
FROM hr.employees ORDER BY WEEKS_WORKED ;
```

Output:

LAST_NAME	WEEKS_WORKED
Popp	29
Mourgos	29
Markle	29
Gee	29
Philtanker	29
Zlotkey	29
Tuvault	29

3. Study the NULLIF Function and solve the following question:

Write a query to display: employee_id, salary, commission_pct, total_earnings, which is calculated as: total_earnings = salary + (salary×commission_pct) If commission_pct is NULL, it should be treated as 0 to prevent calculation errors.

Query:

```
SELECT EMPLOYEE_ID , SALARY, COMMISSION_PCT ,
SALARY + ( SALARY * NULLIF( COMMISSION_PCT , 0 ) ) AS
TOTAL_EARNINGS
FROM hr.employees ; -- Can use NVL also.
```

Output:

142	3100	-	-
143	2600	-	-
144	2500	-	-
145	14000	.4	19600
146	13500	.3	17550
147	12000	.3	15600
148	11000	.3	14300
149	10500	.2	12600

4. Display each employee's last name, hire date, and salary review date, which is the first Monday after six months of service. Label the column REVIEW. Format the dates to appear similar to "Monday, the Thirty-First of July, 2000.

Query:

```
SELECT LAST_NAME, HIRE_DATE,
('Monday, the ' ||
TO_CHAR( NEXT_DAY( ADD_MONTHS(HIRE_DATE, 6), 'MONDAY'), 'DD') ||
CASE
    WHEN TO_NUMBER( TO_CHAR( NEXT_DAY( ADD_MONTHS( HIRE_DATE, 6 ), 'MONDAY' ), 'DD' )
) IN (1, 21, 31) THEN 'st'
    WHEN TO_NUMBER( TO_CHAR( NEXT_DAY( ADD_MONTHS( HIRE_DATE, 6 ), 'MONDAY' ), 'DD' )
) IN (2, 22) THEN 'nd'
    WHEN TO_NUMBER( TO_CHAR( NEXT_DAY( ADD_MONTHS( HIRE_DATE, 6 ), 'MONDAY' ), 'DD' )
) IN (3, 23) THEN 'rd'
    ELSE 'th'
END
|| ' of ' ||
TO_CHAR( NEXT_DAY( ADD_MONTHS(HIRE_DATE, 6), 'MONDAY' ), 'FMMonth,YYYY') ) AS REVIEW
FROM hr.employees;
```

Output:

LAST_NAME	HIRE_DATE	REVIEW
King	17-JUN-03	Monday, the 22nd of December,2003
Kochhar	21-SEP-05	Monday, the 27th of March,2006
De Haan	13-JAN-01	Monday, the 16th of July,2001
Hunold	03-JAN-06	Monday, the 10th of July,2006
Ernst	21-MAY-07	Monday, the 26th of November,2007
Austin	25-JUN-05	Monday, the 26th of December,2005
Pataballa	05-FEB-06	Monday, the 07th of August,2006

5. Create a query to display the first name and salary for all employees. Format the salary to be 15 characters long, right-padded with &. Label the column SALARY

Query:

```
SELECT FIRST_NAME, RPAD(SALARY, 15, '$') AS SALARY FROM hr.employees;
```

Output:

FIRST_NAME	SALARY
Steven	24000$$$$$$$$$$
Neena	17000$$$$$$$$$$
Lex	17000$$$$$$$$$$
Alexander	9000$$$$$$$$$$$
Bruce	6000$$$$$$$$$$$
David	4800$$$$$$$$$$$
Valli	4800$$$$$$$$$$$

6. Write a query that produces the following for each employee: earns monthly but wants . Label the column Dream Salaries.

Query:

```
SELECT ( LAST_NAME || ' earns ' || SALARY || ' monthly '
|| 'but wants ' || SALARY * 3 ) AS DREAM_SALARY
FROM hr.employees;
```

Output:

DREAM_SALARY
King earns 24000 monthly but wants 72000
Kochhar earns 17000 monthly but wants 51000
De Haan earns 17000 monthly but wants 51000
Hunold earns 9000 monthly but wants 27000
Ernst earns 6000 monthly but wants 18000
Austin earns 4800 monthly but wants 14400
Pataballa earns 4800 monthly but wants 14400

7. Create a query that displays the employees' last names and indicates the amounts of their annual salaries with asterisks. Each asterisk signifies a thousand dollars. Sort the data in descending order of salary. Label the column EMPLOYEES_AND_THEIR_SALARIES.

Query:

```
SELECT LAST_NAME,
RPAD('*', ROUND(SALARY / 1000), '*') AS
EMPLOYEES_AND_THEIR_SALARIES -- Show salary with * also.
FROM hr.employees ORDER BY SALARY DESC;
```

Output:

LAST_NAME	EMPLOYEES_AND_THEIR_SALARIES
King	**********
Kochhar	**********
De Haan	**********
Russell	**********
Partners	**********
Hartstein	**********
Greenberg	**********

8. Solve first 2 questions of “Advanced String Functions / Regex / Clause” section of SQL 50 Badge on leetcode.

You can verify my progress and review the questions I've solved by visiting my LeetCode profile: https://leetcode.com/u/obaid4601261/

Lab-04 (DBMS)

SATURDAY, FEBRUARY 8, 2025

NED University
Name: MUHAMMAD OBAID
Roll Number : CT-23025

Note: The results are not displayed in full due to the large number of rows.

Exercise:

2. Study Self Join and solve the following question: Display the employee last name and employee id along with their manager's last name and manager id. Label the columns as Employee_ID, Emp_LN , Manager_ID, and Mgr_LN.

Query:

```
SELECT e.EMPLOYEE_ID , e.LAST_NAME AS EMP_LN ,
       m.EMPLOYEE_ID AS Manager_ID , m.LAST_NAME AS Mgr_LN
FROM hr.employees e LEFT JOIN hr.employees m ON e.MANAGER_ID =
m.EMPLOYEE_ID ;
```

Output:

EMPLOYEE_ID	EMP_LN	MANAGER_ID	MGR_LN
100	King	-	-
101	Kochhar	100	King
102	De Haan	100	King
103	Hunold	102	De Haan
104	Ernst	103	Hunold
105	Austin	103	Hunold
106	Pataballa	103	Hunold

3. Write a query to display the last name, job, department number, and department name for all employees who work in Toronto.

Query:

```sql
SELECT e.LAST_NAME, j.JOB_TITLE, d.DEPARTMENT_ID,
d.DEPARTMENT_NAME, l.CITY
FROM hr.employees e
JOIN hr.jobs j ON e.JOB_ID = j.JOB_ID
JOIN hr.departments d ON d.DEPARTMENT_ID = e.DEPARTMENT_ID
JOIN hr.locations l ON l.LOCATION_ID = d.LOCATION_ID
WHERE l.CITY = 'Toronto' ;
```

Output:

LAST_NAME	JOB_TITLE	DEPARTMENT_ID	DEPARTMENT_NAME	CITY
Hartstein	Marketing Manager	20	Marketing	Toronto
Fay	Marketing Representative	20	Marketing	Toronto

Download CSV

2 rows selected.

4. Display the employee full name and department name for all employees who have an i (lowercase) in their full names.

Query:

```sql
SELECT e.LAST_NAME, d.DEPARTMENT_NAME FROM hr.employees e
JOIN hr.departments d ON e.DEPARTMENT_ID = d.DEPARTMENT_ID
WHERE e.LAST_NAME LIKE '%i%' ;
```

Output:

LAST_NAME	DEPARTMENT_NAME
Hartstein	Marketing
Tobias	Purchasing
Baida	Purchasing
Himuro	Purchasing
Mavris	Human Resources
Davies	Shipping
Dellinger	Shipping

5. Write a query to display the employee last name, department name, location ID, and city of all employees who earn a commission.

Query:

```
SELECT e.LAST_NAME, d.DEPARTMENT_NAME, l.LOCATION_ID , l.CITY,
e.COMMISSION_PCT
FROM hr.employees e
JOIN hr.departments d ON e.DEPARTMENT_ID = d.DEPARTMENT_ID
JOIN hr.locations l ON l.LOCATION_ID = d.LOCATION_ID
WHERE e.COMMISSION_PCT IS NOT NULL ;
```

Output:

LAST_NAME	DEPARTMENT_NAME	LOCATION_ID	CITY	COMMISSION_PCT
Russell	Sales	2500	Oxford	.4
Partners	Sales	2500	Oxford	.3
Errazuriz	Sales	2500	Oxford	.3
Cambrault	Sales	2500	Oxford	.3
Zlotkey	Sales	2500	Oxford	.2
Tucker	Sales	2500	Oxford	.3
Bernstein	Sales	2500	Oxford	.25

6. Display the names and hire dates for all employees who were hired before their managers, along with their manager's names and hire dates. Label the columns Employee, Emp_Hired, Manager, and Mgr_Hired. Order the results in descending by the employee number.

Query:

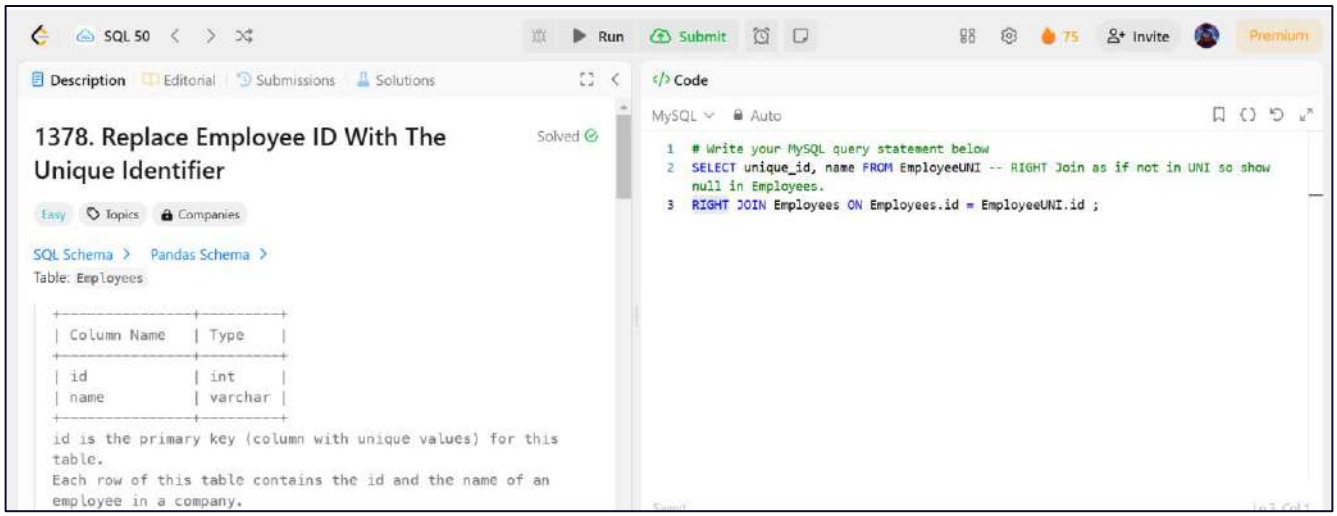
```
SELECT e.FIRST_NAME || ' ' || e.LAST_NAME AS Employee,
e.HIRE_DATE AS Emp_Hired ,
       m.FIRST_NAME || ' ' || m.LAST_NAME AS Manager,
m.HIRE_DATE AS Mgr_Hired
FROM hr.employees e
LEFT JOIN hr.employees m ON e.MANAGER_ID = m.EMPLOYEE_ID
        AND e.HIRE_DATE < m.HIRE_DATE -- If compared in WHERE-
clause then NULL Will be lost.
ORDER BY e.EMPLOYEE_ID DESC ;
```

Output:

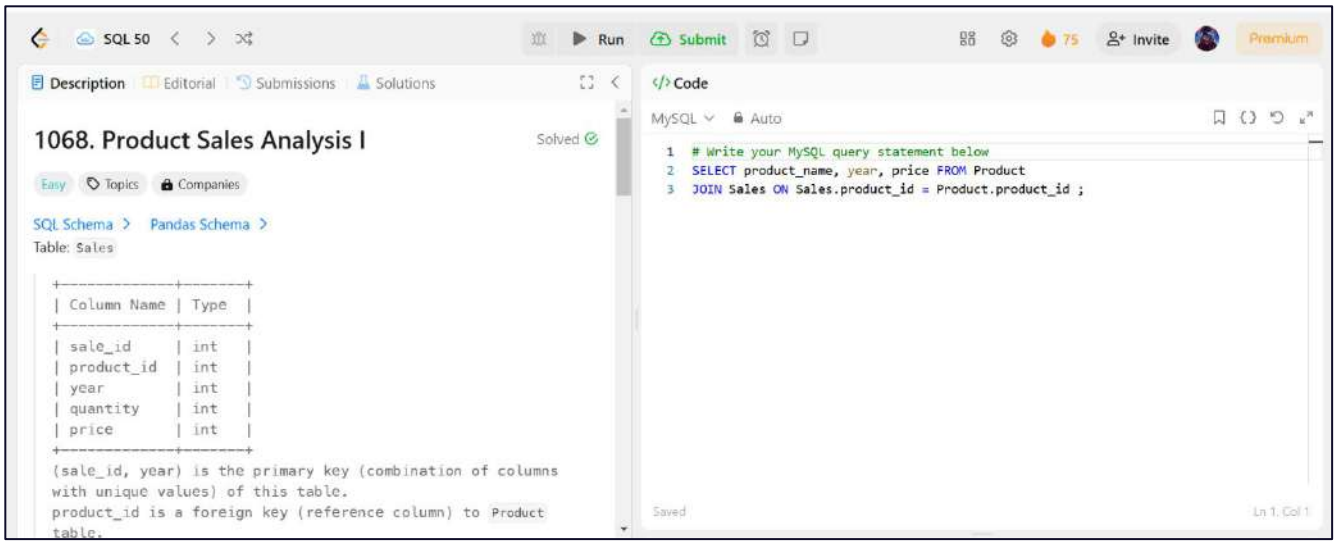
EMPLOYEE	EMP_HIRED	MANAGER	MGR_HIRED
William Gietz	07-JUN-02		-
Shelley Higgins	07-JUN-02	Neena Kochhar	21-SEP-05
Hermann Baer	07-JUN-02	Neena Kochhar	21-SEP-05
Susan Mavris	07-JUN-02	Neena Kochhar	21-SEP-05
Pat Fay	17-AUG-05		-
Michael Hartstein	17-FEB-04		-
Jennifer Whalen	17-SEP-03	Neena Kochhar	21-SEP-05

1. Complete the 'Basic Joins' section of SQL 50 Badge on Leetcode.

Q1:



Q2:



Q3:

SQL 50 | Run | Submit | 75 | Invite | Premium

Description | Editorial | Submissions | Solutions

1581. Customer Who Visited but Did Not Make Any Transactions

Solved

Easy | Topics | Companies

SQL Schema > Pandas Schema >

Table: Visits

Column Name	Type
visit_id	int
customer_id	int

visit_id is the column with unique values for this table. This table contains information about the customers who visited the mall.

Code

MySQL | Auto

```
# Write your MySQL query statement below
-- SELECT customer_id , COUNT( customer_id ) AS count_no_trans
-- FROM Visits WHERE Visits.visit_id NOT IN ( SELECT visit_id FROM Transactions )
-- GROUP BY Visits.customer_id ;

SELECT v.customer_id, COUNT(v.customer_id) AS count_no_trans
FROM Visits v
LEFT JOIN Transactions t ON v.visit_id = t.visit_id
WHERE t.visit_id is NULL -- Only give notVisited once.
GROUP BY v.customer_id ;
```

Saved | Ln 1, Col 1

Q4:

SQL 50 | Run | Submit | 75 | Invite | Premium

Description | Editorial | Submissions | Solutions

197. Rising Temperature

Solved

Easy | Topics | Companies

SQL Schema > Pandas Schema >

Table: Weather

Column Name	Type
id	int
recordDate	date
temperature	int

id is the column with unique values for this table. There are no different rows with the same recordDate. This table contains information about the temperature on a certain day.

Code

MySQL | Auto

```
# Write your MySQL query statement below

-- -- MY METHOD.
-- SELECT w1.id FROM Weather w1
-- JOIN Weather w2 ON w1.temperature > w2.temperature
-- WHERE w1.recordDate - 1 = w2.recordDate ;

SELECT w1.id FROM Weather w1
-- JOIN Weather w2 ON dateDIFF( w1.recordDate, w2.recordDate ) = 1 -- OR
JOIN Weather w2 ON w1.recordDate = DATE_ADD(w2.recordDate, Interval 1 day)
WHERE w1.temperature > w2.temperature ;
```

Saved | Ln 1, Col 1

Q5:

SQL 50 | Run | Submit | 75 | Invite | Premium

Description | Editorial | Submissions | Solutions

1661. Average Time of Process per Machine

Solved

Easy | Topics | Companies

SQL Schema > Pandas Schema >

Table: Activity

Column Name	Type
machine_id	int
process_id	int
activity_type	enum
timestamp	float

The table shows the user activities for a factory website. (machine_id, process_id, activity_type) is the primary key (combination of columns with unique values) of this table.

Code

MySQL | Auto

```
-- # Write your MySQL query statement below
SELECT a1.machine_id, ROUND( AVG( a2.timestamp - a1.timestamp ), 3) AS processing_time FROM Activity a1
JOIN Activity a2 WHERE a1.machine_id = a2.machine_id AND a1.process_id = a2.process_id AND a1.activity_type = 'start' AND a2.activity_type = 'end'
GROUP BY a1.machine_id ;

-- SELECT a1.machine_id,
--        ROUND( AVG(
--            ( SELECT a2.timestamp FROM Activity a2
--              WHERE a1.machine_id = a2.machine_id
--               AND a1.process_id = a2.process_id
--               AND a2.activity_type = 'end'
--             LIMIT 1) - a1.timestamp ), 3 ) AS processing_time -- This will mimic as BruteForce.
-- FROM Activity a1
-- WHERE a1.activity_type = 'start'
-- GROUP BY a1.machine_id ;
```

Saved | Ln 1, Col 1

Q6:

SQL 50 | Run | Submit | 75 | Invite | Premium

Description | Editorial | Submissions | Solutions

577. Employee Bonus

Solved

Easy | Topics | Companies | Hint

SQL Schema > Pandas Schema >

Table: Employee

Column Name	Type
empId	int
name	varchar
supervisor	int
salary	int

empId is the column with unique values for this table.
Each row of this table indicates the name and the ID of an employee in addition to their salary and the id of their manager.

Code

MySQL | Auto

```mysql
# Write your MySQL query statement below
SELECT e.name, b.bonus FROM Employee e
LEFT JOIN Bonus b ON b.empId = e.empId
WHERE b.bonus < 1000 OR b.bonus is NULL ;
```

Saved | Ln 3, Col 40

Q7:

SQL 50 | Run | Submit | 75 | Invite | Premium

Description | Editorial | Submissions | Solutions

1280. Students and Examinations

Solved

Easy | Topics | Companies

SQL Schema > Pandas Schema >

Table: Students

Column Name	Type
student_id	int
student_name	varchar

student_id is the primary key (column with unique values) for this table.
Each row of this table contains the ID and the name of one student in the school.

Code

MySQL | Auto

```mysql
# Write your MySQL query statement below
SELECT s.student_id , s.student_name, sub.subject_name , COUNT( e.subject_name )
as attended_exams FROM Students s
CROSS JOIN Subjects sub -- As we need to find each combination
LEFT JOIN Examinations e on s.student_id = e.student_id AND sub.subject_name = e.
subject_name
GROUP BY s.student_id, s.student_name, sub.subject_name -- Do separately for
each-student, his subject and ID.
ORDER BY s.student_id, sub.subject_name ;
```

Saved | Ln 1, Col 1

Q8:

SQL 50 | Run | Submit | 75 | Invite | Premium

Description | Editorial | Submissions | Solutions

570. Managers with at Least 5 Direct Reports

Solved

Medium | Topics | Companies | Hint

SQL Schema > Pandas Schema >

Table: Employee

Column Name	Type
id	int
name	varchar
department	varchar
managerId	int

id is the primary key (column with unique values) for this table.
Each row of this table indicates the name of an employee.

Code

MySQL | Auto

```mysql
# Write your MySQL query statement below
-- Runtime Error.
-- SELECT name FROM Employee WHERE id =
--     ( SELECT managerId FROM Employee
--       GROUP BY managerId HAVING( COUNT( managerId ) >= 5 ) ) ;

-- 1st Method; As we need to compare with all id's so use IN for list NOT =.
Also apply COUNT() on whole group instead of using WHERE.
-- SELECT name FROM Employee WHERE id IN
--     ( SELECT managerId FROM Employee
--       GROUP BY managerId HAVING( COUNT( managerId ) >= 5 ) ) ;

-- 2nd Method; Using SELF JOIN.
SELECT e.name FROM Employee e
JOIN (
    SELECT managerId FROM Employee
    GROUP BY managerId HAVING COUNT(managerId) >= 5
) m ON e.id = m.managerId;
```

Saved | Ln 1, Col 1

Q9:

SQL 50 | Run | Submit | 75 | Invite | Premium

Description | Editorial | Submissions | Solutions

1934. Confirmation Rate

Solved

Medium | Topics | Companies

SQL Schema > Pandas Schema >

Table: `Signups`

```
+----------------+----------+
| Column Name    | Type     |
+----------------+----------+
| user_id        | int      |
| time_stamp     | datetime |
+----------------+----------+
```

user_id is the column of unique values for this table.
Each row contains information about the signup time for the user with ID user_id.

Code

MySQL | Auto

```mysql
# Write your MySQL query statement below
SELECT s.user_id, IFNULL( ROUND( COUNT(
CASE
    WHEN c.action = "Confirmed" then 1
    -- ELSE 0 -- Don't use 0 as we don't want the "rows" except our condition to return anything.
END
) / COUNT( c.action ) , 2 ), 0 ) as confirmation_rate
FROM Signups s
LEFT JOIN Confirmations c ON c.user_id = s.user_id
GROUP BY s.user_id ;
```

Lab-05
(DBMS)

MONDAY, FEBRUARY 17, 2025

NED University
Name: MUHAMMAD OBAID
Roll Number : CT-23025

Note: The results are not displayed in full due to the large number of rows.

Exercise:

1. Display the highest, lowest, sum, and average salary of all employees. Label the columns Maximum, Minimum , Sum , and Average , respectively. Round your results to the nearest whole number.

Source Code:

```
SELECT MAX( SALARY ) AS Maximum , MIN( SALARY ) AS Minimum ,
       SUM( SALARY ) AS Sum , ROUND( AVG( SALARY ) ) AS Average FROM hr.employees ;
```

Output:

Maximum	Minimum	Sum	Average
24000	2100	692016	6467

Download CSV

2. Modify the query in Q1 to display the minimum, maximum, sum, and average salary for each job type.

Source Code:

```
SELECT JOB_ID, MAX( SALARY ) AS Maximum , MIN( SALARY ) AS Minimum ,
SUM( SALARY ) AS Sum , ROUND( AVG( SALARY ) ) AS Average
FROM hr.employees GROUP BY JOB_ID ;
```

Output:

JOB_ID	MAXIMUM	MINIMUM	SUM	AVERAGE
AD_VP	17000	17000	34000	17000
FI_ACCOUNT	9000	6900	39600	7920
PU_CLERK	3100	2500	13900	2780
SH_CLERK	4200	2500	64300	3215
HR_REP	6500	6500	6500	6500
PU_MAN	11000	11000	11000	11000
AC_MGR	12008	12008	12008	12008
ST_CLERK	3600	2100	55700	2785

3. Write a query to display the number of people with the same job.

Source Code:

```
SELECT JOB_ID, COUNT(*) AS People_Employeed
FROM hr.employees GROUP BY JOB_ID ;
```

Output:

JOB_ID	PEOPLE_EMPLOYEED
AC_ACCOUNT	1
AC_MGR	1
AD_ASST	1
AD_PRES	1
AD_VP	2
FI_ACCOUNT	5
FI_MGR	1
HR_REP	1

4. Write a query to display each department's name, location, number of employees, and the average salary for all employees in that department. Label the columns Name , Location , Number of People , and Salary , respectively. Round the average salary to two decimal places.

Source Code:

```
SELECT d.DEPARTMENT_NAME AS Name , l.CITY || l.STATE_PROVINCE AS Location ,
COUNT( e.EMPLOYEE_ID ) AS No_of_people , ROUND( AVG( e.SALARY ), 2 ) AS SALARY FROM hr.departments d
JOIN hr.locations l ON l.LOCATION_ID = d.LOCATION_ID
JOIN hr.employees e ON e.DEPARTMENT_ID = d.DEPARTMENT_ID
GROUP BY d.DEPARTMENT_NAME , l.CITY,  l.STATE_PROVINCE ;
```

Output:

NAME	LOCATION	NO_OF_PEOPLE	SALARY
Purchasing	SeattleWashington	6	4150
Shipping	South San FranciscoCalifornia	45	3475.56
Human Resources	London	1	6500
Public Relations	MunichBavaria	1	10000
IT	SouthlakeTexas	5	5760
Finance	SeattleWashington	6	8601.33
Executive	SeattleWashington	3	19333.33
Marketing	TorontoOntario	2	9500

5. Display the manager number and the salary of the lowest paid employee for that manager. Exclude anyone whose manager is not known. Exclude any groups where the minimum salary is less than $6,000. Sort the output in descending order of salary.

Source Code:

```
SELECT e.MANAGER_ID , MIN( e.SALARY ) AS min_salary FROM hr.employees e
JOIN hr.employees m ON e.MANAGER_ID = m.EMPLOYEE_ID AND e.SALARY >= 6000
GROUP BY e.MANAGER_ID
ORDER BY min_salary DESC ;
```

Output:

MANAGER_ID	MIN_SALARY
102	9000
205	8300
145	7000
146	7000
108	6900
100	6500
101	6500
147	6200

6. Create a query that will display the total number of employees and, of that total, the number of employees hired in 2005, 2006, and 2007. Create appropriate column headings.

Source Code:

```
SELECT
   COUNT(*) AS total_employees,
   COUNT( CASE WHEN TO_CHAR( hire_date, 'YYYY' ) = 2005 THEN 1 END ) AS employees_hired_in_2005,
   COUNT( CASE WHEN TO_CHAR( hire_date, 'YYYY' ) = 2006 THEN 1 END ) AS employees_hired_in_2006,
   COUNT( CASE WHEN EXTRACT(YEAR FROM hire_date) = 2007 THEN 1 END ) AS employees_hired_in_2007
FROM hr.employees;
```

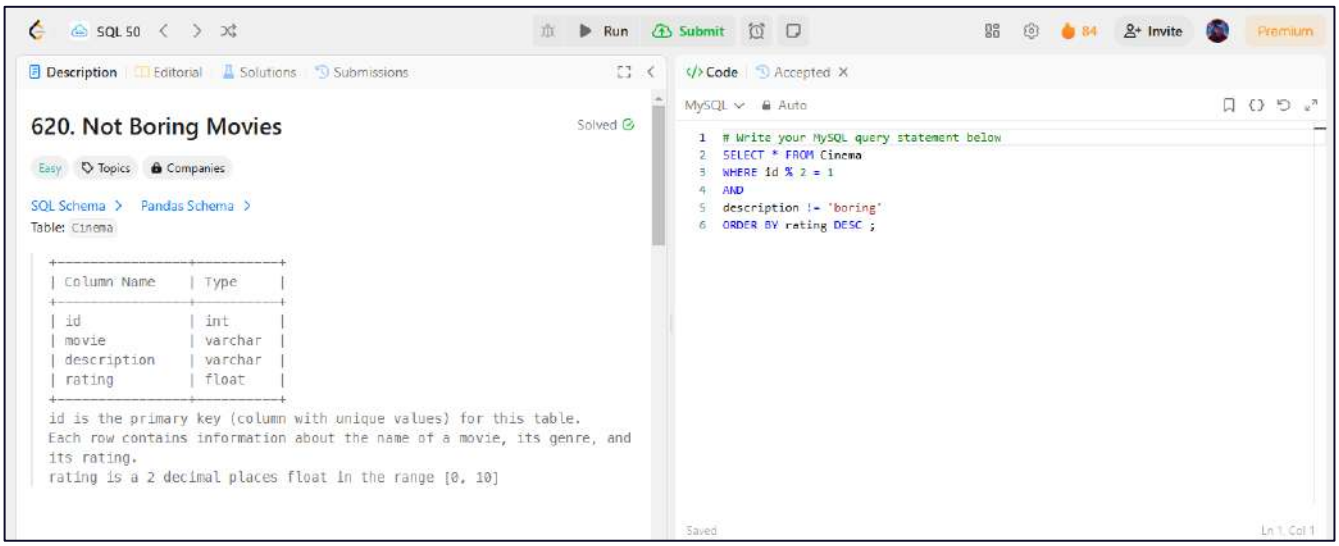
Output:

TOTAL_EMPLOYEES	EMPLOYEES_HIRED_IN_2005	EMPLOYEES_HIRED_IN_2006	EMPLOYEES_HIRED_IN_2007
107	29	24	19

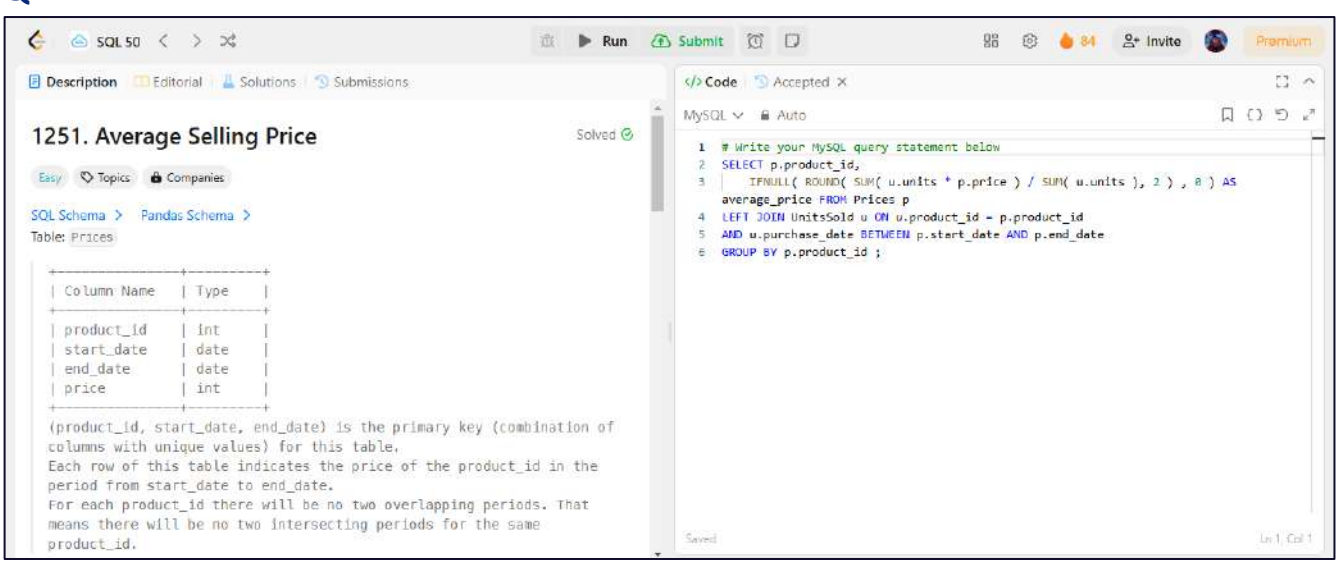
Download CSV

7. Complete the section “Basic Aggregate Functions” of SQL 50 Badge on Leetcode.

Q1:



Q2 :



Q3 :

SQL 50 | Run | Submit | 84 | Invite | Premium

Description | Editorial | Solutions | Submissions

1075. Project Employees I

Solved

Easy | Topics | Companies

SQL Schema > Pandas Schema >

Table: Project

```
+-------------+---------+
| Column Name | Type    |
+-------------+---------+
| project_id  | int     |
| employee_id | int     |
+-------------+---------+
(project_id, employee_id) is the primary key of this table.
employee_id is a foreign key to Employee table.
Each row of this table indicates that the employee with employee_id is
working on the project with project_id.
```

Table: Employee

```
+------------------+---------+
```

Code | Accepted

MySQL | Auto

```mysql
# Write your MySQL query statement below
-- SELECT p.project_id, ROUND( SUM( p.employee_id ) / COUNT( p.employee_id ) , 2 )
SELECT p.project_id, ROUND( AVG( e.experience_years ), 2 )
    AS average_years
FROM Employee e
JOIN Project p ON p.employee_id = e.employee_id
GROUP BY p.project_id ;
```

Saved | Ln 1, Col 1

Q4 :

SQL 50 | Run | Submit | 84 | Invite | Premium

Description | Editorial | Solutions | Submissions

1633. Percentage of Users Attended a Contest

Solved

Easy | Topics | Companies

SQL Schema > Pandas Schema >

Table: Users

```
+-------------+---------+
| Column Name | Type    |
+-------------+---------+
| user_id     | int     |
| user_name   | varchar |
+-------------+---------+
user_id is the primary key (column with unique values) for this table.
Each row of this table contains the name and the id of a user.
```

Table: Register

```
+-------------+---------+
| Column Name | Type    |
+-------------+---------+
```

Code

MySQL | Auto

```mysql
# Write your MySQL query statement below
SELECT contest_id,
ROUND( COUNT( DISTINCT user_id ) / ( SELECT COUNT( user_id ) FROM Users ) * 100.
0 , 2 ) AS percentage
FROM Register
GROUP BY contest_id
ORDER BY percentage DESC , contest_id ;

-- Using Joins.
-- SELECT
--     r.contest_id,
--     ROUND(COUNT(DISTINCT r.user_id) * 100 / u.total_users, 2) AS percentage
-- FROM
--     Register r
-- JOIN
--     (SELECT COUNT(user_id) AS total_users FROM Users) u ON 1=1 -- Join on all
where condition met.
-- GROUP BY
--     r.contest_id
-- ORDER BY
--     percentage DESC, r.contest_id;
```

Saved | Ln 1, Col 1

Q5 :

SQL 50 | Run | Submit | 84 | Invite | Premium

Description | Editorial | Solutions | Submissions

1211. Queries Quality and Percentage

Solved

Easy | Topics | Companies

SQL Schema > Pandas Schema >

Table: Queries

Column Name	Type
query_name	varchar
result	varchar
position	int
rating	int

This table may have duplicate rows.
This table contains information collected from some queries on a database.
The position column has a value from **1** to **500**.
The rating column has a value from **1** to **5**. Query with rating less than 3 is a poor query.

Code

MySQL | Auto

```mysql
# Write your MySQL query statement below
SELECT query_name , ROUND( AVG( rating / position ) , 2 ) AS quality -- Its same as SUM( rating / position ) / COUNT( position )
,
ROUND( SUM( CASE WHEN rating < 3 THEN 1 ELSE 0 END ) * 100 / COUNT(*) , 2 ) AS poor_query_percentage -- Here we need to "SUM" else Wrong ans.
FROM Queries
GROUP BY query_name ;
```

Saved | Ln 1, Col 1

Q6 :

SQL 50 | Run | Submit | 84 | Invite | Premium

Description | Editorial | Solutions | Submissions

1193. Monthly Transactions I

Solved

Medium | Topics | Companies

SQL Schema > Pandas Schema >

Table: Transactions

Column Name	Type
id	int
country	varchar
state	enum
amount	int
trans_date	date

id is the primary key of this table.
The table has information about incoming transactions.
The state column is an enum of type ["approved", "declined"].

Code

MySQL | Auto

```mysql
# Write your MySQL query statement below
SELECT DATE_FORMAT( trans_date, '%Y-%m') AS month, country ,
COUNT( * ) AS trans_count,
SUM( CASE WHEN state = 'approved' THEN 1 ELSE 0 END ) AS approved_count ,
SUM( amount ) AS trans_total_amount ,
SUM( CASE WHEN state = 'approved' THEN amount ELSE 0 END ) as approved_total_amount
FROM Transactions
GROUP BY month, country ;
```

Saved | Ln 1, Col 1

Q7 :

SQL 50 | Run | Submit | 84 | Invite | Premium

Description | Editorial | Solutions | Submissions

1174. Immediate Food Delivery II

Solved

Medium | Topics | Companies

SQL Schema > Pandas Schema >

Table: Delivery

Column Name	Type
delivery_id	int
customer_id	int
order_date	date
customer_pref_delivery_date	date

delivery_id is the column of unique values of this table.
The table holds information about food delivery to customers that make orders at some date and specify a preferred delivery date (on the same order date or after it).

Code

MySQL | Auto

```sql
# Write your MySQL query statement below
SELECT ROUND( AVG( order_date =  customer_pref_delivery_date ) * 100 , 2 )
AS immediate_percentage
FROM Delivery
WHERE ( customer_id , order_date ) IN ( SELECT customer_id , MIN( order_date ) FROM
Delivery GROUP BY customer_id ) ; -- Since it return > 1 row SO use IN.
```

Saved | Ln 1, Col 1

Q8 :

SQL 50 | Run | Submit | 84 | Invite | Premium

Description | Editorial | Solutions | Submissions

550. Game Play Analysis IV

Solved

Medium | Topics | Companies

SQL Schema > Pandas Schema >

Table: Activity

Column Name	Type
player_id	int
device_id	int
event_date	date
games_played	int

(player_id, event_date) is the primary key (combination of columns with unique values) of this table.
This table shows the activity of players of some games.
Each row is a record of a player who logged in and played a number of games (possibly 0) before logging out on someday using some device.

Code

MySQL | Auto

```sql
# Write your MySQL query statement below
SELECT ROUND( COUNT( a1.player_id ) / ( SELECT COUNT( DISTINCT a3.player_id ) FROM
Activity a3 ), 2) AS fraction
FROM Activity a1
WHERE ( a1.player_id , DATE_SUB( a1.event_date, INTERVAL 1 DAY ) ) -- Use DATE_SUB
not add as we need to find all upcoming dates consecutive to 1st login date.
    IN ( SELECT a2.player_id, MIN( a2.event_date ) FROM Activity a2
        GROUP BY a2.player_id );
```

Saved | Ln 1, Col 1

Lab-06 (DBMS)

WEDNESDAY, APRIL 9, 2025

NED University
Name: MUHAMMAD OBAID
Roll Number : CT-23025

Note: The results are not displayed in full due to the large number of rows.

Exercise:

1. Write a query to display the last name and hire date of any employee in the same department as Zlotkey. Exclude Zlotkey.

Query:

```
SELECT LAST_NAME, HIRE_DATE FROM hr.employees
WHERE DEPARTMENT_ID = ( SELECT DEPARTMENT_ID FROM hr.employees
                        WHERE LOWER( LAST_NAME ) = 'zlotkey' )
 -- <> for NOT-EQUAL.
AND LOWER( LAST_NAME ) <> 'zlotkey' ;
```

Output:

LAST_NAME	HIRE_DATE
Russell	01-OCT-04
Partners	05-JAN-05
Errazuriz	10-MAR-05
Cambrault	15-OCT-07
Tucker	30-JAN-05
Bernstein	24-MAR-05
Hall	20-AUG-05

2. Create a query to display the employee numbers and last names of all employees who earn more than the average salary. Sort the results in ascending order of salary.

Query:

```sql
SELECT EMPLOYEE_ID, LAST_NAME, SALARY FROM hr.employees
WHERE SALARY > ( SELECT AVG( SALARY ) FROM hr.employees )
ORDER BY SALARY ;
```

Output:

EMPLOYEE_ID	LAST_NAME	SALARY
203	Mavris	6500
123	Vollman	6500
165	Lee	6800
113	Popp	6900
155	Tuvault	7000
161	Sewall	7000
178	Grant	7000

3. Write a query that displays the employee numbers and last names of all employees who work in a department with any employee whose last name contains an ‘e’.

Query:

```sql
SELECT EMPLOYEE_ID, LAST_NAME, DEPARTMENT_ID FROM hr.employees
WHERE DEPARTMENT_ID IN ( SELECT DISTINCT DEPARTMENT_ID FROM
hr.employees -- Used DISTINCT to avoid duplicates.
                        WHERE LAST_NAME LIKE '%e%' ) ;
```

Output:

EMPLOYEE_ID	LAST_NAME	DEPARTMENT_ID
145	Russell	80
146	Partners	80
147	Errazuriz	80
148	Cambrault	80
149	Zlotkey	80
150	Tucker	80
151	Bernstein	80

4. Display the last name, department number, and job ID of all employees whose department location ID is 1700.

Query:

```
SELECT LAST_NAME, DEPARTMENT_ID, JOB_ID FROM hr.employees
WHERE DEPARTMENT_ID IN ( SELECT DEPARTMENT_ID FROM
hr.departments
                        WHERE LOCATION_ID = 1700 ) ;
```

Output:

LAST_NAME	DEPARTMENT_ID	JOB_ID
King	90	AD_PRES
Kochhar	90	AD_VP
De Haan	90	AD_VP
Greenberg	100	FI_MGR
Faviet	100	FI_ACCOUNT
Chen	100	FI_ACCOUNT
Sciarra	100	FI_ACCOUNT

5. Display the last name and salary of every employee who does not report to King.

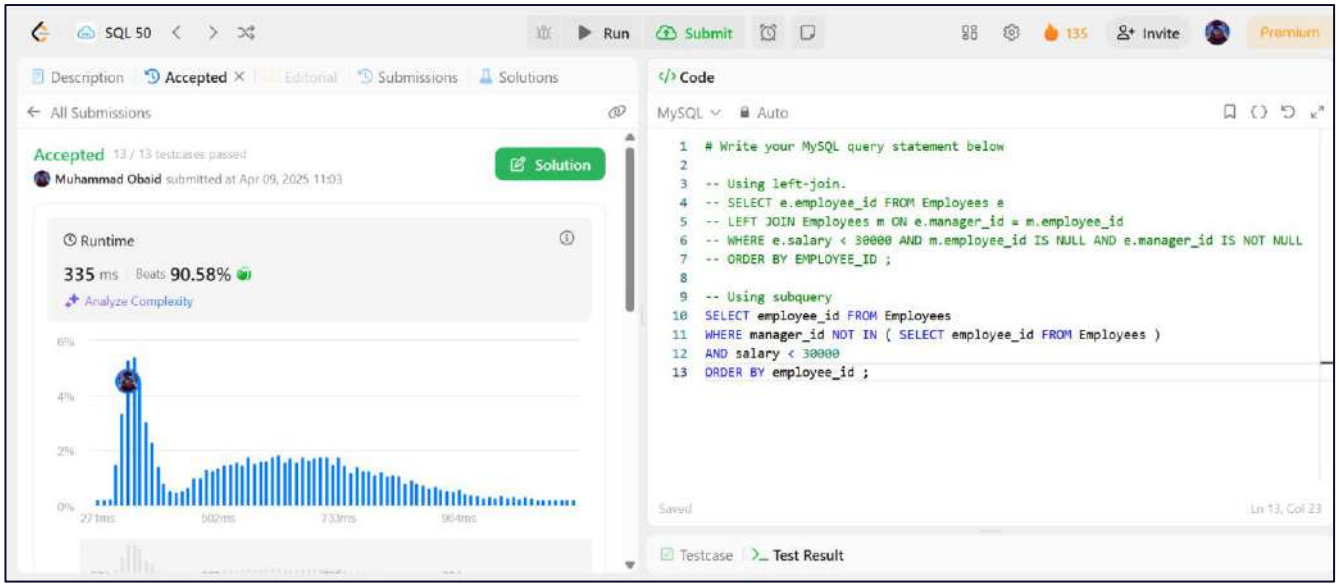
Query:

```
SELECT LAST_NAME, SALARY FROM hr.employees
WHERE MANAGER_ID NOT IN ( SELECT EMPLOYEE_ID FROM hr.employees
WHERE LOWER( LAST_NAME ) = 'king' ) ;
```

Output:

LAST_NAME	SALARY
Kochhar	17000
De Haan	17000
Raphaely	11000
Weiss	8000
Fripp	8200
Kaufling	7900
Vollman	6500

Leetcode Subqueries Section:



SQL 50

Description | Accepted | Editorial | Submissions | Solutions

All Submissions

Accepted 14 / 14 testcases passed

Muhammad Obaid submitted at Apr 09, 2025 12:16

Editorial | Solution

Runtime

385 ms | Beats 74.48%

Analyze Complexity

Code | Code Sample

MySQL | Auto

```mysql
# Write your MySQL query statement below
SELECT s.id,
CASE
    WHEN s.id % 2 = 1 THEN IFNULL( ( SELECT student FROM Seat WHERE id = s.id + 1 ), student ) -- Handle null-case use COALESCE or IFNULL for "odd".
    ELSE ( SELECT student FROM Seat WHERE id = s.id - 1 )
END
AS student
FROM Seat s ;
```

Saved | Ln 4, Col 14

SQL 50

Description | Accepted | Editorial | Submissions | Solutions

All Submissions

Accepted 19 / 19 testcases passed

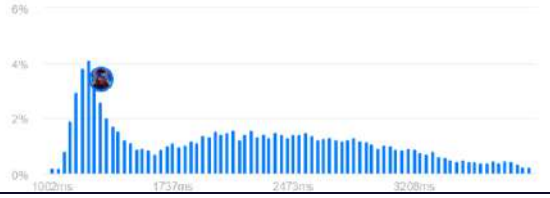
Muhammad Obaid submitted at Apr 09, 2025 18:03

Solution

Runtime

1318 ms | Beats 79.66%

Analyze Complexity



Code | Code Sample

MySQL | Auto

```mysql
# Write your MySQL query statement below
-- SELECT u_name AS results
-- FROM (
--     SELECT u.name AS u_name , COUNT(m.user_id) AS rating_count
--     FROM Users u
--     JOIN MovieRating m ON u.user_id = m.user_id
--     GROUP BY u.name
-- ) AS subquery
-- ORDER BY rating_count DESC , u_name LIMIT 1 ; -- For checking can't do order-by within derived table so do outside.

-- UNION (Removes duplicates) while UNION ALL ( allows )
SELECT name as results
FROM (
    SELECT u.name FROM Users u
    JOIN MovieRating m ON u.user_id = m.user_id
    GROUP BY u.name
    ORDER BY COUNT(m.user_id) DESC, u.name LIMIT 1
```

Saved | Ln 23, Col 7

SQL 50

Description | Accepted | Editorial | Submissions | Solutions

All Submissions

Accepted 14 / 14 testcases passed

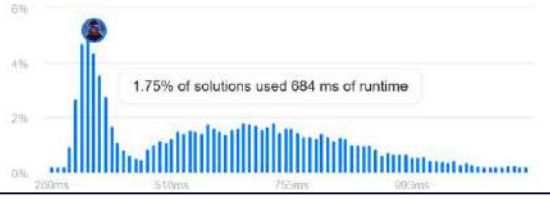
Muhammad Obaid submitted at Apr 09, 2025 18:28

Solution

Runtime

373 ms | Beats 80.78%

Analyze Complexity



Code

MySQL | Auto

```mysql
# Write your MySQL query statement below
SELECT c.visited_on,
( SELECT SUM(amount) FROM Customer
  WHERE visited_on BETWEEN DATE_SUB(c.visited_on, INTERVAL 6 DAY) AND
  c.visited_on ) -- Without "c.visited_on" condition it was giving runtime error. Also simple c.visited_on WILL help us filter-out rows WHILE "IS NOT NULL" condition can conflict with b/w ONE giving wrong answer.
AS amount,
ROUND(
( SELECT SUM(amount) / 7 FROM Customer
  WHERE visited_on BETWEEN DATE_SUB(c.visited_on, INTERVAL 6 DAY) AND
  c.visited_on ) , 2 )
AS average_amount
FROM Customer c
WHERE visited_on >= ( SELECT MIN( visited_on ) + INTERVAL 6 DAY FROM Customer )
GROUP BY c.visited_on ORDER BY c.visited_on ;
```

Saved | Ln 14, Col 45

SQL 50

Description | Accepted | Editorial | Submissions | Solutions

All Submissions

Accepted 12 / 12 testcases passed

Muhammad Obaid submitted at Apr 09, 2025 20:10

Runtime

302 ms | Beats 91.71%

Analyze Complexity

```mysql
# Write your MySQL query statement below
SELECT requester_id as id , COUNT(*) AS num
FROM ( SELECT requester_id FROM RequestAccepted
UNION ALL
SELECT accepter_id FROM RequestAccepted ) AS count -- Count how many times it appears in both columns.
GROUP BY id ORDER BY num DESC LIMIT 1 ;
```

Testcase | Test Result

Accepted Runtime: 67 ms

Case 1

Input

RequestAccepted =

requester_id	accepter_id	accept_date

SQL 50

Description | Accepted | Editorial | Submissions | Solutions

All Submissions

Accepted 21 / 21 testcases passed

Muhammad Obaid submitted at Apr 09, 2025 21:13

Runtime

554 ms | Beats 81.91%

Analyze Complexity

```mysql
# Write your MySQL query statement below
SELECT ROUND( SUM( tiv_2016 ), 2 ) AS tiv_2016 FROM Insurance
WHERE tiv_2015 IN ( SELECT tiv_2015 FROM Insurance
                    GROUP BY tiv_2015 HAVING COUNT( * ) > 1 ) -- Having duplicates so count > 1.
AND ( lat , lon ) IN ( SELECT lat, lon FROM Insurance
                       GROUP BY lat, lon HAVING COUNT(*) = 1 ) ;

-- SELECT ROUND( SUM(tiv_2016), 2 ) AS tiv_2016 FROM Insurance i1
-- where i1.tiv_2015 IN ( SELECT i2.tiv_2015 FROM Insurance i2 WHERE i1.pid != i2.pid )
-- AND ( lat,lon ) NOT IN ( SELECT i2.lat ,i2.lon FROM Insurance i2 WHERE i1.pid != i2.pid ) ; -- Directly compare id's instead of individual lat, lons
```

SQL 50

Description | Accepted | Editorial | Submissions | Solutions

All Submissions

Accepted 21 / 21 testcases passed

Muhammad Obaid submitted at Apr 09, 2025 21:26

Runtime

1172 ms | Beats 76.33%

Analyze Complexity

```mysql
# Write your MySQL query statement below
-- SELECT d.name AS Department , e1.name AS Employee , e1.salary AS Salary
-- FROM Employee e1 JOIN Department d ON e1.departmentId = d.id
-- WHERE ( SELECT COUNT( DISTINCT e2.salary ) FROM Employee e2
--         WHERE e2.salary > e1.salary AND e1.departmentId = e2.departmentId ) < 3 ; -- Get top3 DISTINCT salaries.

SELECT d.name AS Department , e1.name AS Employee , e1.salary AS Salary
FROM Employee e1 JOIN Department d ON e1.departmentId = d.id
JOIN Employee e2 ON e1.departmentId = e2.departmentId AND e1.salary <= e2.salary
GROUP BY e1.id
HAVING COUNT( DISTINCT e2.salary ) <= 3 ;
```

Lab-07,8 (DBMS)

SUNDAY, 27 APRIL, 2025

NED University
Name: MUHAMMAD OBAID
Roll Number : CT-23025

Note: The results are not displayed in full due to the large number of rows.

Exercise:

1. Create MY_EMPLOYEE table having attributes employee_id, first_name, last_name, userid, salary. Choose appropriate data types for each field. Make employee_id as primary key.

```
CREATE TABLE MY_EMPLOYEE (
    employee_id INT PRIMARY KEY,
    first_name VARCHAR2(50) NOT NULL,
    last_name VARCHAR2(50) NOT NULL,
    userid VARCHAR2(30) UNIQUE NOT NULL,
    salary NUMBER(10, 2) NOT NULL
);
```

2. Rename MY_EMPLOYEE table as MY_STAFF.

Query:

```
ALTER TABLE MY_EMPLOYEE RENAME TO MY_STAFF ;
```

Output:

```
SQL> ALTER TABLE MY_EMPLOYEE RENAME TO MY_STAFF



Table MY_EMPLOYEE altered.

Elapsed: 00:00:00.011
```

3. Describe the structure of the MY_STAFF table to identify the column names.

Query:

```
DESC MY_STAFF ;
```

Output:

```
Name        Null?    Type
----------- -------- ------------
EMPLOYEE_ID NOT NULL NUMBER(38)
FIRST_NAME  NOT NULL VARCHAR2(50)
LAST_NAME   NOT NULL VARCHAR2(50)
USERID      NOT NULL VARCHAR2(30)
SALARY      NOT NULL NUMBER(10,2)
```

4. Add the first row of data to the MY_STAFF table from the following sample data. Do not list the columns in the INSERT clause.

```
INSERT INTO MY_STAFF
VALUES (1, 'Ralph', 'Patel', 'rpatel', 895) ;
```

5. Populate the MY_STAFF table with the second row of sample data from the preceding list. This time, list the columns explicitly in the INSERT clause.

```
INSERT INTO MY_STAFF (employee_id, first_name, last_name, userid, salary)
VALUES (2, 'Betty', 'Dancs', 'bdancs', 860);
```

6. Confirm your addition to the table.

Download Execution time: 0.005 seconds

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	USERID
1	1	Ralph	Patel	rpatel
2	2	Betty	Dancs	bdancs

7. Populate the table with the next two rows of sample data by running the INSERT statement in such a way that you concatenate the first letter of the first name and the first seven characters of the last name to produce the user ID, instead of hardcoding it

```
INSERT INTO MY_STAFF
VALUES (3, 'Ben', 'Biri',
        CONCAT(UPPER(SUBSTR('Ben', 1, 1)), LOWER(SUBSTR('Biri', 1, 7))),
        1100);

INSERT INTO MY_STAFF
VALUES (4, 'Chad', 'Newman',
        UPPER(SUBSTR('Chad', 1, 1)) || LOWER(SUBSTR('Newman', 1, 7)),
        750);
```

8. Confirm your additions to the table and make the data additions permanent.

Query:

```
SELECT * FROM MY_STAFF ;
COMMIT ;
```

Output:

Query result | Script output | DBMS output | Explain Plan | SQL history

Download ▾ Execution time: 0.002 seconds

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	USERID	SALARY
1	1	Ralph	Patel	rpatel	
2	2	Betty	Dancs	bdancs	
3	4	Chad	Newman	Cnewman	
4	3	Ben	Biri	Bbiri	

9. Change the last name of employee 3 to Drexler.

```
UPDATE MY_STAFF SET last_name = 'Drexler' WHERE employee_id = 3 ;
```

10. Change the salary to 1000 for all employees with a salary less than 900. Verify your changes to the table.

Query:

```
UPDATE MY_STAFF SET salary = 1000 WHERE SALARY < 900 ;
```

Output:

	'EE_ID	FIRST_NAME	LAST_NAME	USERID	SALARY
2	2	Betty	Dancs	bdancs	1000
3	4	Chad	Newman	Cnewman	1000
4	3	Ben	Drexler	Bbiri	1100

11. Delete Betty Dancs from the MY_STAFF table.

```
DELETE FROM MY_STAFF WHERE ( FIRST_NAME || ' ' || LAST_NAME ) = 'Betty Dancs' ;
```

12. Confirm your changes to the table and Commit all pending changes.

	YEE_ID	FIRST_NAME	LAST_NAME	USERID	SALARY
1	1	Ralph	Patel	rpatel	1000
2	4	Chad	Newman	Cnewman	1000
3	3	Ben	Drexler	Bbiri	1100

13. Populate the table with the last row of sample data by modifying the insert statement that you created in step 7. Confirm your addition to the table. Create savepoint.

Query:

```
INSERT INTO MY_STAFF
VALUES (5, 'Audrey', 'Ropeburn',
        UPPER(SUBSTR('Audrey', 1, 1)) || LOWER(SUBSTR('Ropeburn', 1, 7)),
        1550);
SAVEPOINT mySVP ;
```

Output:

	'EE_ID	FIRST_NAME	LAST_NAME	USERID	SALARY
1	1	Ralph	Patel	rpatel	1000
2	4	Chad	Newman	Cnewman	1000
3	5	Audrey	Ropeburn	Aropebur	1550
4	3	Ben	Drexler	Bbiri	1100

14. Empty the entire table and confirm that the table is empty.

Query:

```
DELETE FROM MY_STAFF ;
```

Output:

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	USERID	SALARY
No items to display.				

15. Discard the most recent DELETE operation without discarding the earlier INSERT operation. Confirm that the new row is still intact. Make the data addition now permanent.

```
ROLLBACK TO mySVP ;
COMMIT;
```

It will give error like this as for savepoint in LiveOracle Sql all changes must be done in 1 transaction with savepoint.

```
Error starting at line : 4 in command -
ROLLBACK TO mySVP
Error report -
ORA-01086: savepoint mySVP never established
01086. 00000 -  "savepoint '%s' never established in this session or is invalid"
*Cause:    An attempt was made to roll back to a savepoint that was never
           established in this session, or was invalid.
*Action:   Try rolling back to the savepoint from the session where it is established.
```

16. Create MY_DEPART table with necessary attributes. Choose appropriate data types for each field. Make depart_id as primary key.

```
CREATE TABLE MY_DEPART (
    depart_id NUMBER PRIMARY KEY,
    depart_name VARCHAR2(100) NOT NULL,
    location VARCHAR2(100),
    manager_id NUMBER,
    CONSTRAINT fk_manager FOREIGN KEY (manager_id) REFERENCES MY_STAFF (employee_id)
);
```

17. Modify MY_STAFF table to include depart_id as foreign key. What happens to previous 5 rows with this change in the MY_STAFF table?

```
ALTER TABLE MY_STAFF ADD depart_id NUMBER;

ALTER TABLE MY_STAFF
ADD CONSTRAINT fk_depart_id FOREIGN KEY (depart_id)
REFERENCES MY_DEPART (depart_id);
```

When you add a foreign key constraint to an existing table like `MY_STAFF`, it enforces referential integrity. Specifically, the foreign key ensures that any value in `depart_id` of `MY_STAFF` must exist in `depart_id` of `MY_DEPART`.

Scenario 1: If `depart_id` is `NULL` for the Previous Rows:

- **No issues**: If the `depart_id` column is set to `NULL` for the existing rows in `MY_STAFF` (i.e., no value for `depart_id`), no error occurs when you add the foreign key constraint because `NULL` is always allowed in a foreign key column. These rows will have `depart_id` set to `NULL` unless you explicitly assign them a valid value.

Scenario 2: If `depart_id` is not `NULL` but doesn't match any `depart_id` in `MY_DEPART`:

- **Foreign Key Violation**: If there are existing rows in `MY_STAFF` that have a `depart_id` that doesn't exist in `MY_DEPART`, **Oracle will throw an error** when trying to add the foreign key constraint. This is because the database will not allow a foreign key that references a non-existent value in the referenced table (`MY_DEPART`).

- **Fix**: You will need to ensure that the `depart_id` values in `MY_STAFF` already exist in `MY_DEPART` or set them to `NULL` before adding the constraint.

Scenario 3: If `depart_id` values are already correct (existing rows match a `depart_id` in `MY_DEPART`):

- **No issues**: If the `depart_id` values in `MY_STAFF` already match valid `depart_id` values in `MY_DEPART`, adding the foreign key constraint will work without any issues. The rows will remain intact, and the referential integrity will be enforced.

Lab-09 (DBMS)

SUNDAY, MAY 4, 2025

NED University
Name: MUHAMMAD OBAID
Roll Number : CT-23025

Note: The results are not displayed in full due to the large number of rows.

Exercise:

1. In the MY_STAFF table that you created in the last lab, Add a COMMISSION column of NUMBER data type, precision 2, scale 2. Add a constraint to the commission column that ensures that a commission value is greater than zero.

Query:

```
ALTER TABLE MY_STAFF ADD (
    COMMISSION NUMBER(2, 2),
    CONSTRAINT chk_comm_pos CHECK (COMMISSION >
0);
```

Output:

```
Table MY_STAFF altered.

Elapsed: 00:00:00.040
```

2. Create a table STAFF_UPDATES with the same structure as MY_STAFF. Keep it unconstrained i.e. do not define the primary key and foreign key constraints in this table. Populate it with the following data:

employee_id	first_name	last_name	userid	salary	depart_id	Commission
1	Ralph	Patel	rpatel	920	101	0.05
6	Zara	Lee	zlee	1100	103	0.01

Query:

```sql
-- Create STAFF_UPDATES without copying constraints,
-- 1=0 ensures that no-row OR constraints are copied.
CREATE TABLE STAFF_UPDATES AS
SELECT employee_id, first_name, last_name, userid, salary, depart_id, CAST(NULL AS NUMBER(4,2)) AS
commission
FROM MY_STAFF
WHERE 1 = 0 ;

INSERT INTO STAFF_UPDATES (employee_id, first_name, last_name, userid, salary, depart_id, commission)
VALUES (1, 'Ralph', 'Patel', 'rpatel', 920, 101, 0.05);

INSERT INTO STAFF_UPDATES (employee_id, first_name, last_name, userid, salary, depart_id, commission)
VALUES (6, 'Zara', 'Lee', 'zlee', 1100, 103, 0.01);
```

3. Give logical reason why we made MY_STAFF table unconstrained.

Reason:

The logical reason for making the STAFF_UPDATES table unconstrained (not MY_STAFF, but STAFF_UPDATES) is:

To allow staging for temporary or intermediate data changes for testing purpose where we might receive partial or invalid data (e.g. missing foreign keys, duplicate IDs) during batch weekly updates in a business without being restricted by constraints such as primary keys, foreign keys or any other business rules.

After the data is staged, it can be cleaned and validated and once all inconsistencies are resolved, the data can then be securely merged into the constrained STAFF table, ensuring integrity without violating business rules after each month.

4. Insert a few departments in MY_DEPART (that you created in the last lab session), (e.g., 101 – HR, 102 – Sales, 103 – IT) and update the existing staff to be part of one of these departments.

Query:

```sql
INSERT INTO MY_DEPART (depart_id, depart_name, location, manager_id)
VALUES (101, 'HR', 'New York', NULL);

INSERT INTO MY_DEPART (depart_id, depart_name, location, manager_id)
VALUES (102, 'Sales', 'Chicago', NULL);

INSERT INTO MY_DEPART (depart_id, depart_name, location, manager_id)
VALUES (103, 'IT', 'San Francisco', NULL);

UPDATE MY_STAFF SET depart_id = 101 WHERE employee_id = 1;

UPDATE MY_STAFF SET depart_id = 103 WHERE employee_id = 3;
```

Output:

	ME	USERID	SALARY	DEPART_ID	COMMISSION
1		Bbiri	1100	103	*(null)*
2		rpatel	895	101	*(null)*

5. Use a MERGE statement to update existing records in MY_STAFF if employee_id matches, and insert new rows if they don't exist. Verify the changes in MY_STAFF.

Query:

```
MERGE INTO MY_STAFF target
USING STAFF_UPDATES src
ON ( target.employee_id = src.employee_id )
WHEN MATCHED THEN
    UPDATE SET
        target.first_name = src.first_name,
        target.last_name = src.last_name,
        target.userid = src.userid,
        target.salary = src.salary,
        target.depart_id = src.depart_id,
        target.commission = src.commission
WHEN NOT MATCHED THEN
    INSERT (employee_id, first_name, last_name, userid, salary, depart_id,
commission)
    VALUES (src.employee_id, src.first_name, src.last_name, sr
```

Output:

	ME	USERID	SALARY	DEPART_ID	COMMISSION
1		Bbiri	1100	103	*(null)*
2		zlee	1100	103	0.01
3		rpatel	920	101	0.05

6. Alter the MY_STAFF table to add a foreign key constraint on depart_id that references the MY_DEPART table with ON DELETE CASCADE.

Query:

```
-- ON DELETE CASCADE: ensures that when a record in parent table is deleted ; its also deleted from
child-relational table.
ALTER TABLE MY_STAFF DROP CONSTRAINT fk_depart_id ;
ALTER TABLE MY_STAFF
ADD CONSTRAINT fk_depart_id FOREIGN KEY (depart_id)
REFERENCES MY_DEPART (depart_id) ON DELETE CASCADE;
```

7. Delete department 103 and verify that all staff associated with it are automatically removed from MY_STAFF.

Query:

```
DELETE FROM MY_DEPART WHERE depart_id = 103;
SELECT * FROM MY_STAFF ;
```

Output:

Query result | Script output | DBMS output | Explain Plan | SQL history

Download ▾ Execution time: 0.005 seconds

	ME	USERID	SALARY	DEPART_ID	COMMISSION
1		rpatel	920	101	0.05

8. Drop the foreign key constraint on depart_id, and recreate it using ON DELETE SET NULL.

Query:

```
ALTER TABLE MY_STAFF DROP CONSTRAINT fk_depart_id ;
ALTER TABLE MY_STAFF
ADD CONSTRAINT fk_depart_id FOREIGN KEY (depart_id)
REFERENCES MY_DEPART (depart_id) ON DELETE SET NULL ;
```

Output:

```
Table MY_STAFF altered.

Elapsed: 00:00:00.017
```

9. Insert a department 104 – "Finance" and assign it to a new staff member.

Query:

```
INSERT INTO MY_DEPART (depart_id, depart_name, location,
manager_id)
VALUES (104, 'Finance', 'Boston', NULL);
INSERT INTO MY_STAFF (employee_id, first_name, last_name,
userid, salary, depart_id, commission)
VALUES (7, 'Liam', 'Turner', 'lturner', 950, 104, 0.03);
```

Output:

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	USERID	SALARY
1	7	Liam	Turner	lturner	
2	1	Ralph	Patel	rpatel	

10. Delete department 104 and confirm that the depart_id of affected staff is now set to NULL.

Query:

```
DELETE FROM MY_DEPART WHERE depart_id = 104;
SELECT * FROM MY_STAFF ;
```

Output:

	ME	USERID	SALARY	DEPART_ID	COMMISSION
1		lturner	950	*(null)*	0.03
2		rpatel	920	101	0.05

11. Create an audit table named STAFF_AUDIT with the following columns:
(employee_id NUMBER, action_type VARCHAR2(10), action_date DATE)

Query:

```
CREATE TABLE STAFF_AUDIT (
    employee_id NUMBER,
    action_type VARCHAR2(10),
    action_date DATE
);
```

12. Create a trigger on MY_STAFF that fires AFTER INSERT OR DELETE and records the employee_id, action type ('INSERT' or 'DELETE'), and current date into STAFF_AUDIT.

Query:

```
CREATE OR REPLACE TRIGGER trg_staff_audit
AFTER INSERT OR DELETE ON MY_STAFF
FOR EACH ROW
BEGIN
    IF INSERTING THEN
        INSERT INTO STAFF_AUDIT (employee_id, action_type, action_date)
        VALUES (:NEW.employee_id, 'INSERT', SYSDATE);

    ELSIF DELETING THEN
        INSERT INTO STAFF_AUDIT (employee_id, action_type, action_date)
        VALUES (:OLD.employee_id, 'DELETE', SYSDATE);
    END IF;
END;
```

13. Test the trigger by inserting and deleting a record in MY_STAFF. Query STAFF_AUDIT to verify entries.

Query:

```
INSERT INTO MY_STAFF (employee_id, first_name, last_name, userid, salary,
depart_id, commission)
VALUES (10, 'Alice', 'Smith', 'asmith', 1500, 101, 0.06);
INSERT INTO MY_STAFF (employee_id, first_name, last_name, userid, salary,
depart_id, commission)
VALUES (11, 'Bob', 'Johnson', 'bjohnson', 1300, 102, 0.04);
INSERT INTO MY_STAFF (employee_id, first_name, last_name, userid, salary,
depart_id, commission)
VALUES (12, 'Charlie', 'Brown', 'cbrown', 1400, 101, 0.05);

DELETE FROM MY_STAFF WHERE employee_id = 10;
SELECT * FROM STAFF_AUDIT ;
```

Output:

	EMPLOYEE_ID	ACTION_TYPE	ACTION_DATE
1	12	INSERT	5/3/2025, 4:38:33 PM
2	10	DELETE	5/3/2025, 4:38:53 PM
3	10	INSERT	5/3/2025, 4:37:53 PM
4	11	INSERT	5/3/2025, 4:38:03 PM